

Arquitectura de Microservicios con Contenedores

Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Demo: construyendo un container

Namespaces · cgroups · OverlayFS · runc – paso a paso

Objetivos de la demo

Al terminar vas a poder:

- **Reproducir** lo que hace Docker con primitivas del kernel
- **Explicar** cada namespace y cada controller de cgroup v2
- **Ver** el output real de cada aislamiento
- **Entender** OverlayFS, capabilities, seccomp y el bundle OCI
- **Armar** el ciclo de vida create → start → stop → delete

El lab

9 fases · 21 pasos · sin Docker: solo `unshare`, `chroot/pivot_root`, `ip netns`, `cgroups` y `runc`.

Fuente: `linux-containers/building-docker`

Mapa de la demo

Fase	Qué aísla / limita
1	Filesystem (chroot, mount ns, pivot_root)
2	Procesos (PID ns)
3	Red (net ns, bridge, veth)
4	Identidad (UTS, IPC)
5	Usuarios (user ns)
6	Recursos (cgroup v2)
7	Seguridad (capabilities, seccomp)
8	Capas + runtime (OverlayFS, OCI/runc)
9	Todo junto + lifecycle

Cómo leer cada slide

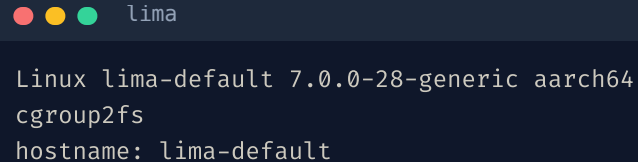
Para cada paso verás:

1. **Concepto** – qué primitiva del kernel
2. **Comando | Output** – izquierda: qué ejecutamos; derecha (con click): captura real en Lima
3. **Qué estamos viendo** – interpretación

Entorno (Lima)

```
# En el Mac - lima es alias de limactl shell default:
lima                # shell interactiva en la VM
lima sudo uname -a   # un comando puntual
lima sudo bash -c '...' # varios comandos
```

Capturado en esta máquina:



```
lima
Linux lima-default 7.0.0-28-generic aarch64
cgroup2fs
hostname: lima-default
```

Setup previo (una vez)

```
# 1) User namespaces (Ubuntu 23.10+)
echo 0 | sudo tee /proc/sys/kernel/apparmor_restrict_unprivileged_userns

# 2) Bridge / forwarding (Fase 3)
echo 0 | sudo tee /proc/sys/net/bridge/bridge-nf-call-iptables 2>/dev/null || true
sudo iptables -P FORWARD ACCEPT 2>/dev/null || true
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

Sin (1), unshare --user falla con Operation not permitted.

Cleanup entre pasos

```
rm -rf /tmp/container_* /tmp/overlay /tmp/oci-bundle /tmp/test_*  
ip netns list | awk '{print $1}' | xargs -r -I{} ip netns delete {}  
ip link del br0 2>/dev/null || true  
# cgroups hoja vacíos:  
rmdir /sys/fs/cgroup/memtest /sys/fs/cgroup/cputest 2>/dev/null || true
```

Fase 1 – Filesystem

`chroot · mount namespace · pivot_root`

Step 1.1 – Concepto: chroot

chroot cambia la raíz aparente (/) de un proceso: los paths absolutos se resuelven dentro de un árbol que armamos a mano.

No crea namespaces: solo aísla el filesystem. Procesos, red y mounts siguen siendo los del host.

Step 1.1 – Preparar rootfs + entrar

```
mkdir -p /tmp/container_root/{bin,etc,proc,sys,dev,lib}
# copiar bash, ls, cat, ... + libs con ldd
# (en Ubuntu 24.04: symlink usr/bin → ../bin)

sudo chroot /tmp/container_root /bin/bash
```

Esperado: `pwd` → `/`; `ls /` solo muestra lo copiado;
`ls /home` falla.

```
pwd          # /
ls /         # bin etc lib proc ...
ls /home     # No such file or directory
whoami       # root
```

Limitación (montar `/proc` desde afuera):

```
# en el host:
mount -t proc proc /tmp/container_root/proc
# adentro: ps aux → aún lista procesos del HOST
```

Step 1.1 – Qué estamos viendo

-  Aislamiento de **paths** (/ ya no es el del host)
-  Sin mount ns: montar /proc afecta el mismo mount table
-  Sin PID ns: ps ve todo el host
-  Sin net ns: `ip addr` muestra interfaces del host

chroot es el ancestro pedagógico; Docker usa `pivot_root` + namespaces.

Step 1.2 – Concepto: mount namespace

Un **mount namespace** da a cada proceso su propia tabla de montajes.

Lo que montás adentro **no** aparece en el host (propagación default: `private`).

```
sudo unshare --mount --fork bash
```

Step 1.2 – Aislar un tmpfs

```
# dentro del unshare:
readlink /proc/self/ns/mnt
mkdir -p /tmp/test_mount
mount -t tmpfs tmpfs /tmp/test_mount
echo "Hello from mount namespace" > /tmp/test_mount/test.txt

# en otra terminal (host):
mount | grep test_mount
cat /tmp/test_mount/test.txt
```

Esperado: IDs distintos; en el host el mount y el archivo **no existen**.

```
lima · mount namespace

host mnt: mnt:[4026531832]
ns mnt:   mnt:[4026532347]
inside: tmpfs on /tmp/test_mount type tmpfs (...)
Hello from mount namespace

host sees mount? (empty – isolated)
host cat: No such file or directory
```

El archivo vive en un filesystem montado **solo** en ese namespace.

Step 1.2 – Combinar con chroot

```
mount -t proc proc /tmp/container_root_v2/proc
mount -t sysfs sysfs /tmp/container_root_v2/sys
chroot /tmp/container_root_v2 /bin/bash
# adentro: ls /proc funciona
# pero ps aún muestra procesos del host (falta PID ns)
```

Mount ns aísla mounts; **no** aísla la lista de procesos.

Step 1.3 – Concepto: pivot_root

`pivot_root` mueve la raíz vieja a `.oldroot` y la desmonta. Es lo que usa Docker.

	chroot	pivot_root
Raíz vieja	accesible	desmontada
Mount ns	no requerido	sí
Escape con <code>..</code>	posible	no

Requisito: la nueva raíz debe ser un **mount point** (`mount --bind` sobre sí misma).

Step 1.3 – Comando

```
sudo unshare --mount --fork bash
mount --bind /tmp/container_root_pivot /tmp/container_root_pivot
cd /tmp/container_root_pivot
mkdir -p .oldroot
pivot_root . .oldroot
cd /
mount -t proc proc /proc
umount -l /.oldroot && rmdir /.oldroot
exec /bin/bash
```

Esperado: `ls /` solo rootfs del container; `ls /.oldroot` falla.

Step 1.3 – Qué estamos viendo

1. Bind mount → cumple el requisito de mount point
2. `pivot_root` intercambia raíces
3. Unmount de `.oldroot` → sin escape al host
4. Todavía falta PID ns: `ps` puede ver el host

Fase 2 – Procesos

PID namespace (+ mount)

Step 2.1 – Concepto: PID namespace

El PID namespace numera procesos desde **1**.

Pero /proc del host sigue mostrando todos los PIDs hasta que lo **remontás** en un mount namespace.

⚠ umount /proc **sin** --mount afecta al host. No lo hagas en demo en vivo.

Step 2.1 – PID + mount

```
sudo unshare --pid --mount --fork bash
umount -l /proc 2>/dev/null || true
mount -t proc proc /proc
echo $$           # → 1
ps aux           # solo procesos del namespace
```

--fork es obligatorio con --pid.

```
lima · unshare --pid --mount --fork
```

```
host pid ns: pid:[4026531836]
pid ns:      pid:[4026532479]
shell_pid=1
--- ps aux ---
USER  PID  COMMAND
root   1  bash -c ...
root   6  ps aux
--- after sleep ---
root   1  bash ...
root   7  sleep 1000
```

Adentro: pocos procesos, PIDs chicos. En el host el mismo sleep tiene **otro** PID.

Step 2.2 – Qué estamos viendo

- PID ns decide **qué** procesos existen en la vista
- Mount ns decide **qué** /proc se lee
- Remount de /proc es el paso que más se olvida
- El proceso 1 es el "init" del container: reap de zombies

Con filesystem + PID tenemos el núcleo de un container. Falta red.

Fase 3 – Red

Network namespace • veth • bridge

Step 3.1 – Concepto: network namespace

Un net ns aísla interfaces, rutas, iptables y sockets.

Namespace nuevo → solo lo (DOWN). Para conectar al host: par **veth** (cable virtual de dos puntas).

```
ip netns add netns1
ip link add veth0 type veth peer name veth1
ip link set veth1 netns netns1
```

Step 3.1 – veth host ↔ ns

```
ip netns exec netns1 ip link set lo up
ip addr add 10.0.0.1/24 dev veth0 && ip link set veth0 up
ip netns exec netns1 ip addr add 10.0.0.2/24 dev veth1
ip netns exec netns1 ip link set veth1 up

ping -c 2 10.0.0.2
ip netns exec netns1 ping -c 2 10.0.0.1
```

Esperado: ping bidireccional OK.

```
lima · ip netns + veth

netns1 before:
lo                DOWN

netns1 after:
lo                UNKNOWN  127.0.0.1/8
veth1@if13        UP        10.0.0.2/24

PING 10.0.0.2 ... 2 received, 0% packet loss
PING 10.0.0.1 ... 2 received, 0% packet loss
```

El namespace no ve eth0 del host: stack de red propio.

Step 3.1 – Nota (bug del material)

Al conectar un segundo namespace, el peer movido a netns2 suele llamarse veth3.

La IP 10.0.1.2/24 debe ir en **veth3** (lado del ns), no en veth2 (lado host).

```
# correcto:  
ip netns exec netns2 ip addr add 10.0.1.2/24 dev veth3
```

Step 3.2 – Concepto: bridge

Un **bridge** es un switch L2 virtual. Modelo Docker (docker0):

- Contenedores en la misma subred
- Gateway = IP del bridge
- NAT (MASQUERADE) para salir a Internet

Step 3.2 – Bridge

```
ip link add name br0 type bridge
ip addr add 172.16.0.1/16 dev br0 && ip link set br0 up

# veth-c1 / veth-c2 → containers; peers → master br0
ip netns exec container1 ip addr add 172.16.0.10/16 dev veth-
ip netns exec container2 ip addr add 172.16.0.20/16 dev veth-
# default via 172.16.0.1 en ambos

ip netns exec container1 ping -c 2 172.16.0.20
```

Si el ping falla con Docker instalado: `bridge-nf-call-iptables=0` o `iptables -P FORWARD ACCEPT`.

```
lima · bridge br0
```

```
bridge links:
veth-c1-br@... master br0 state forwarding
veth-c2-br@... master br0 state forwarding
```

```
PING 172.16.0.20 ... 2 received, 0% packet loss
```

Dos “containers” en la misma L2, sin rutas estáticas entre ellos: el bridge reenvía frames.

Fase 4 – Identidad

UTS • IPC

Step 4.1 – Concepto: UTS namespace

UTS aísla **hostname** (y NIS domain). Cada container puede llamarse distinto sin tocar el host.

Docker: `docker run --hostname=....` Logs y service discovery usan este nombre.

Step 4.1 – Hostname aislado

```
sudo unshare --uts bash
hostname mycontainer
hostname          # mycontainer

# otra terminal:
hostname          # lima-default (sin cambios)
```

```
lima · unshare --uts

host hostname: lima-default
inside=mycontainer
uts=uts:[4026532664]
host after: lima-default
```

Step 4.2 – Concepto: IPC namespace

IPC aísla shared memory, semáforos y message queues.

Misma key/shmid en dos namespaces = recursos **distintos**. Evita que un container lea la shm de otro.

Step 4.2 – IPC aislado

```
sudo unshare --ipc bash
ipcmk -M 2048 && ipcmk -S 1 && ipcmk -Q
ipcs -a
```

```
# dentro del ns:
Shared memory id: 0
Semaphore id: 0
Message queue id: 0
(ipcs -a muestra los 3 recursos)
```

```
# en el host:
——— Shared Memory Segments ———
(vacío – no ve los del namespace)
```

Fase 5 – Usuarios

User namespace (rootless)

Step 5.1 – Concepto: user namespace

UID 0 ("root") **adentro** se mapea a un UID no privilegiado **en el host**.

Base de containers rootless y de la seguridad: root en el container $\neq$ root en el host.

Step 5.1 – Root adentro, unprivileged afuera

```
# como usuario regular (no sudo su):  
unshare --user --map-root-user bash  
whoami; id; cat /proc/self/uid_map
```

```
lima · unshare --user --map-root-user  
  
whoami=root  
uid=0(root) gid=0(root) ...  
uid_map=  
      0          501          1  
user_ns=user:[4026532664]
```

Adentro: root. En el host: el proceso es UID **501** (fedex).

Step 5.2 – User ns + filesystem

Combinar `--user` `--mount` `--pid` `--fork` `--map-root-user` + `pivot_root`.

Archivos creados como `root:root` adentro aparecen con UID mapeado (ej. 100000) en el host.

Notas:

- Montar `proc/sysfs` en user ns **sin** PID ns → `permission denied`
- Ejecutar como usuario regular para ver el mapping real (como root es 0→0)

Fase 6 – Recursos

cgroup v2: memory · CPU · I/O

Step 6.1 – Concepto: cgroup v2

Jerarquía unificada en `/sys/fs/cgroup` (`cgroup2fs`).

Docker mueve los PIDs del container a un cgroup hoja y escribe límites (`memory.max`, `cpu.max`, ...).

Regla hoja: un cgroup con procesos no puede tener `subtree_control` no vacío.

Step 6.1 – Crear cgroup y mover un proceso

```
stat -fc %T /sys/fs/cgroup          # cgroup2fs
cat /sys/fs/cgroup/cgroup.controllers
mkdir /sys/fs/cgroup/demo_leaf
echo "+memory +cpu" > /sys/fs/cgroup/cgroup.subtree_control
sleep 30 & echo $! > /sys/fs/cgroup/demo_leaf/cgroup.procs
cat /proc/$!/cgroup
```

```
type=cgroup2fs
controllers=cpuset cpu io memory hugetlb pids ...
sleep 22770 → 0::/demo_leaf
```

Step 6.2 – Memory limits

```
mkdir /sys/fs/cgroup/memtest  
echo "8M" > /sys/fs/cgroup/memtest/memory.max  
# proceso ya DENTRO del cgroup alloca > 8M
```

`memory.max` = hard limit. `memory.high` = soft
(throttle antes del OOM).

El proceso muere; `memory.events` muestra:

```
memory.events  
max 38  
oom 1  
oom_kill 1
```

Step 6.3 – CPU limits

```
echo "10000 100000" > /sys/fs/cgroup/cputest/cpu.max # 10%  
# lanzar busy-loop en el cgroup  
cat /sys/fs/cgroup/cputest/cpu.stat
```

```
cpu.stat · 10% quota
```

```
nr_periods 21  
nr_throttled 21  
throttled_usec 1819527
```

Throttling, no kill: el kernel pausa cuando se agota la cuota del periodo.

Step 6.4 – I/O limits

Límites **por dispositivo de bloque** (MAJ:MIN), no por path:

```
DEVICE=$(lsblk -n -o MAJ:MIN,MOUNTPOINT \
| awk '$2="/" {print $1}')
# en Lima fue 253:1 – NO asumir 8:0
echo "$DEVICE rbps=10485760 wbps=5242880" \
> /sys/fs/cgroup/iotest/io.max
```

rbps/wbps = bytes/s de lectura/escritura.

Para **verificar**, el proceso tiene que pegarle al disco de verdad:

```
# proceso YA dentro del cgroup iotest:
dd if=/dev/zero of=/tmp/big.bin bs=1M count=100 \
oflag=direct
```

Sin oflag=direct, dd escribe al **page cache** (RAM): el kernel acepta los bytes al toque y el límite de I/O casi no se nota.

Con oflag=direct, bypassa el cache → cada write va al dispositivo → io.max throttlea de verdad.

Fase 7 – Seguridad

Capabilities · seccomp

Step 7.1 – Capabilities

Root no es todo-o-nada: capabilities granulares (CAP_NET_BIND_SERVICE, ...).

```
capsh --print
capsh --drop=cap_net_bind_service -- -c '
python3 -c "import socket; s=socket.socket(); s.bind(\\", 80))"
'
```

→ Permission denied

setcap solo afecta **binarios ELF**, no scripts Python/Bash.

Docker mantiene ~14 capabilities y dropea el resto.

Step 7.2 – Seccomp

Filtra **syscalls**. Modo estricto: solo read/write/exit/sigreturn.

```
# programa C: prctl(PR_SET_SECCOMP, SECCOMP_MODE_STRICT); fork();  
# → proceso muere al intentar fork
```

Docker aplica un perfil BPF por defecto. Defensa en profundidad junto a namespaces y capabilities.

Fase 8 – Capas y runtime

OverlayFS · OCI / runc

Step 8.1 – OverlayFS

```
mkdir -p /tmp/overlay/{lower,upper,work,merged}
echo "Base file" > /tmp/overlay/lower/base.txt
mount -t overlay overlay \
-o lowerdir=.../lower,upperdir=.../upper,workdir=.../work \
/tmp/overlay/merged
```

Escrituras → **upper** (CoW). Borrados → **whiteouts**.
Lower queda intacto.

```
lima · mount -t overlay

merged=Base file
upper_before=      # vacío
# echo "Modified..." > merged/base.txt
lower_after=Base file # intacto
upper_after=Modified content
# rm merged/base.txt
merged_ls: No such file or directory
lower_still=/tmp/overlay/lower/base.txt
```

Así funcionan los layers de una imagen Docker (overlay2).

Step 8.2 – OCI + runc

Bundle = rootfs/ + config.json. runc aplica namespaces/cgroups/caps por nosotros.

```
mkdir -p /tmp/oci-bundle/rootfs
# copiar bins+libs al rootfs (¡no cp "$lib" "$lib"!)
```

```
cd /tmp/oci-bundle && runc spec
# editar process.args → ["/bin/bash", "-c", "..."]
sudo runc run mycontainer
```

Output (Lima): hello-from-runc (bash corre aislado).

Fase 9 – Todo junto

Script completo · lifecycle

Step 9.1 – Contenedor integrado

Un solo script combina:

- Namespaces: PID, mount, UTS, IPC (, user, net)
- pivot_root + /proc
- cgroup v2: memory.max, cpu.max

```
# paths cgroup v2 (NO /sys/fs/cgroup/memory/... de v1):  
mkdir -p /sys/fs/cgroup/$NAME  
echo "50M" > /sys/fs/cgroup/$NAME/memory.max  
unshare --pid --mount --uts --ipc --fork /tmp/enter_$NAME.sh
```

```
hostname      # nombre del container  
ps aux        # solo procesos del ns  
ls /          # solo rootfs  
cat /sys/fs/cgroup/$NAME/memory.max
```

Equivalente mental a:

```
docker run -it --memory=50m --hostname=...  
alpine sh
```

Step 9 – Lifecycle

Estados: **create** → **start** → **stop** → **delete** (como Docker).

```
/tmp/ctr_manager.sh create test1
/tmp/ctr_manager.sh start test1
/tmp/ctr_manager.sh list          # running + host-pid
/tmp/ctr_manager.sh stop test1
/tmp/ctr_manager.sh delete test1
```

El PID a matar es el del **host** (cgroup.procs), no \$\$ dentro del PID ns (suele ser 1).

Manual	Docker
create + rootfs + cgroup	docker create
unshare + pivot_root	docker start / run
kill host-pid	docker stop
rmdir cgroup + rm rootfs	docker rm
list	docker ps -a

Cierre

De primitivas a Docker

Lo que Docker abstrae

```
docker run
  → containerd
    → runc (OCI)
      → namespaces (mnt/pid/net/uts/ipc/user)
      → cgroups (memory/cpu/io/pids)
      → pivot_root + OverlayFS
      → capabilities + seccomp
```

Hoy recorrimos cada capa **a mano**.

Checklist de aprendizaje

- ■ `chroot` ≠ container
- ■ Mount ns aísla mounts; PID ns + remount /proc aísla procesos
- ■ Net ns + veth + bridge = red de Docker
- ■ User ns: root adentro, unprivileged afuera
- ■ cgroup v2 limita memoria/CPU/I/O
- ■ OverlayFS = layers; runc = runtime OCI

Próximos pasos

1. Repetir los pasos en Lima siguiendo `building-docker/step_XX.md`
2. Comparar con `docker run --memory=... --cpus=... -h ...`
3. Leer `config.json` de un bundle `runc spec`
4. Clase siguiente: imágenes, layers y Dockerfile

Referencias

- Lab: `linux-containers/building-docker`
- `man namespaces`, `man cgroups`, `man unshare`
- OCI Runtime Spec / `runc`
- Kernel docs: `namespaces`, `cgroup-v2`, `overlayfs`